

C# / .NET Learning Roadmap

Topic Notes — Section 13: ASP.NET Core — Web API Fundamentals

Topics: 13.1 through 13.12

Date: 29 June 2026

Section 13 — ASP.NET Core: Web API Fundamentals

This section covers all twelve roadmap topics: 13.1 (Routing), 13.2 (Controllers and actions), 13.3 (Minimal APIs), 13.4 (Model binding and validation), 13.5 (DTOs and mapping), 13.6 (Middleware pipeline), 13.7 (Filters), 13.8 (Content negotiation, status codes, ProblemDetails), 13.9 (API versioning), 13.10 (Swagger/OpenAPI), 13.11 (Health checks), 13.12 (Rate limiting).

13.1. Routing (Attribute Routing, Conventional Routing)

Builds on: 12.6 (hosting -- routing middleware registered with `app.MapControllers()`), 12.2 (DI -- controllers resolved by container).

Attribute routing -- the standard for Web APIs

```
[ApiController]
[Route("api/[controller]")] // [controller] = "orders"
public class OrdersController : ControllerBase
{
    // GET api/orders
    [HttpGet]
    public async Task<IActionResult> GetAll() { ... }

    // GET api/orders/42
    [HttpGet("{id:int}")]
    public async Task<IActionResult> GetById(int id) { ... }

    // GET api/orders/by-customer/Sathish
    [HttpGet("by-customer/{customerName}")]
    public async Task<IActionResult> GetByCustomer(
        string customerName) { ... }

    // POST api/orders
    [HttpPost]
    public async Task<IActionResult> Create(
        [FromBody] CreateOrderDto dto) { ... }

    // PUT api/orders/42
    [HttpPut("{id:int}")]
    public async Task<IActionResult> Update(
        int id, [FromBody] UpdateOrderDto dto) { ... }

    // PATCH api/orders/42/status
    [HttpPatch("{id:int}/status")]
    public async Task<IActionResult> UpdateStatus(
        int id, [FromBody] UpdateStatusDto dto) { ... }

    // DELETE api/orders/42
    [HttpDelete("{id:int}")]
    public async Task<IActionResult> Delete(int id) { ... }
}
```

Route constraints

```
[HttpGet("{id:int}")]      // int only: /orders/42
[HttpGet("{name:alpha}")]  // letters only: /orders/Sathish
[HttpGet("{id:guid}")]     // GUID: /orders/abc-123-...
[HttpGet("{id:min(1)}")]   // int >= 1
[HttpGet("{id:length(5)}")] // exactly 5 chars
[HttpGet("{id:int?}")]     // optional parameter
[HttpGet("{id:int:min(1)}")] // multiple constraints
[HttpGet("**path")]        // catch-all
```

[ApiController] -- what it does

```
[ApiController] // enables:
// 1. Auto model validation -- 400 if ModelState invalid
// 2. Binding source inference -- [FromBody] inferred
// 3. ProblemDetails error responses -- RFC 7807
// 4. Multipart/form-data binding for file uploads
```

Key Takeaways for 13.1

- Attribute routing -- define routes with [HttpGet], [HttpPost], etc.
- [Route("api/[controller]")] -- [controller] replaced with class name minus Controller
- Route constraints {id:int} enforce type matching before action selected
- [ApiController] enables auto-validation, binding inference, ProblemDetails

Debugging Tips

Symptom	Cause	Fix
404 for all routes	app.MapControllers() not called or controller not public	Add app.MapControllers() after middleware; ensure controller is public
Route constraint not matching	Constraint syntax wrong or unsupported type	Check: {id:int}, {id:guid}, {id:alpha} -- see docs for full list
Ambiguous route match exception	Two actions match the same URL pattern	Add unique path segments or constraints to distinguish them
Action method not found	HTTP method mismatch -- POST vs GET	Verify [HttpGet/Post/Put/Delete/Patch] attribute matches client call

13.2. Controllers and Actions

Builds on: 13.1 (routing -- controllers contain actions that routes point to), 12.2 (DI -- controllers constructed by container).

ControllerBase vs Controller

```
// ControllerBase -- for APIs (no view support):
public class OrdersController : ControllerBase { }

// Controller -- for MVC (includes view helpers):
public class HomeController : Controller { }

// APIs should always use ControllerBase
```

Return types -- ActionResult preferred

```
// IActionResult -- flexible but loses type info for Swagger:
[HttpGet("{id:int}")]
public async Task<IActionResult> GetById(int id)
{
    var order = await _service.GetByIdAsync(id);
    if (order is null) return NotFound();
    return Ok(order);
}

// ActionResult<T> -- Swagger knows the response type:
[HttpGet("{id:int}")]
public async Task<ActionResult<OrderDto>> GetById(int id)
{
    var order = await _service.GetByIdAsync(id);
    if (order is null) return NotFound();
    return order;    // implicit conversion to Ok(order)
}
// Prefer ActionResult<T> -- better Swagger documentation
```

Controller helper methods

```
// 2xx Success:
return Ok(data);                // 200 + body
return Created(uri, data);      // 201 + Location
return CreatedAtAction(         // 201 + Location
    nameof(GetById), new { id }, data);
return Accepted();              // 202 async started
return NoContent();             // 204 no body

// 4xx Client errors:
return BadRequest();            // 400
return BadRequest(ModelState);  // 400 + errors
return Unauthorized();          // 401
return Forbid();                // 403
return NotFound();              // 404
return Conflict();              // 409
return UnprocessableEntity();    // 422

// 5xx:
return Problem("Something went wrong"); // 500 ProblemDetails
```

A complete realistic controller

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly IOrderService _service;
    private readonly ILogger<OrdersController> _logger;

    public OrdersController(
        IOrderService service,
```

```

        ILogger<OrdersController> logger)
{ _service = service; _logger = logger; }

[HttpGet]
[ProducesResponseType<PagedResult<OrderDto>>(200)]
public async Task<ActionResult<PagedResult<OrderDto>>>
    GetAll(
        [FromQuery] int page = 1,
        [FromQuery] int pageSize = 20,
        [FromQuery] string? status = null,
        CancellationToken ct = default)
{
    var result = await _service.GetPagedAsync(
        page, pageSize, status, ct);
    return Ok(result);
}

[HttpGet("{id:int}")]
[ProducesResponseType<OrderDto>(200)]
[ProducesResponseType(404)]
public async Task<ActionResult<OrderDto>> GetById(
    int id, CancellationToken ct)
{
    var order = await _service.GetByIdAsync(id, ct);
    return order is null ? NotFound() : Ok(order);
}

[HttpPost]
[ProducesResponseType<OrderDto>(201)]
[ProducesResponseType(400)]
public async Task<ActionResult<OrderDto>> Create(
    CreateOrderDto dto, CancellationToken ct)
{
    var order = await _service.CreateAsync(dto, ct);
    return CreatedAtAction(
        nameof(GetById), new { id = order.Id }, order);
}

[HttpPut("{id:int}")]
[ProducesResponseType(204)]
[ProducesResponseType(404)]
public async Task<IActionResult> Update(
    int id, UpdateOrderDto dto, CancellationToken ct)
{
    var updated = await _service.UpdateAsync(id, dto, ct);
    return updated ? NoContent() : NotFound();
}

[HttpDelete("{id:int}")]
[ProducesResponseType(204)]
[ProducesResponseType(404)]
public async Task<IActionResult> Delete(
    int id, CancellationToken ct)
{
    var deleted = await _service.DeleteAsync(id, ct);

```

```

        return deleted ? NoContent() : NotFound();
    }
}

```

Key Takeaways for 13.2

- ControllerBase for APIs (no views); Controller for MVC
- ActionResult<T> preferred over IActionResult -- Swagger knows response type
- CreatedAtAction(nameof(GetById), new { id }, order) -- correct 201 pattern
- NoContent() (204) for successful updates/deletes with no body
- [ProducesResponseType] decorates expected status codes -- improves Swagger

Debugging Tips

Symptom	Cause	Fix
CreatedAtAction returns 201 but Location header is wrong	Route name or routeValues do not match the target action	Verify nameof() refers to correct action; routeValues keys match params
Action returns 200 but body is empty	Returning Ok() without data, or DTO has no public properties	Pass data to Ok(data); ensure DTO properties have public getters
ModelState.IsValid always true despite invalid data	[ApiController] not applied or validation attributes missing	Add [ApiController] to controller; add [Required] etc. to DTO
Controller constructor throws on resolution	Service not registered in DI	Add builder.Services.AddScoped<IService, Impl>() in Program.cs

13.3. Minimal APIs

Builds on: 13.1 (routing -- minimal APIs define routes inline), 12.6 (hosting -- app.Map*() registers minimal API endpoints).

What are Minimal APIs?

Minimal APIs define HTTP endpoints directly in Program.cs without controllers. Introduced in .NET 6, ideal for microservices and reducing boilerplate.

```

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/api/orders/{id:int}", async (
    int id,
    IOrderService service,
    CancellationToken ct) =>
{
    var order = await service.GetByIdAsync(id, ct);
    return order is null
        ? Results.NotFound()
        : Results.Ok(order);
});

app.Run();

```

Results and TypedResults

```
// Results -- standard response builder:
Results.Ok(data) // 200
Results.Created("/uri", o) // 201 + Location
Results.NoContent() // 204
Results.BadRequest("msg") // 400
Results.NotFound() // 404
Results.Problem("msg") // 500 ProblemDetails

// TypedResults -- preserves type for Swagger:
app.MapGet("/api/orders/{id:int}",
    async Task<Results<Ok<OrderDto>, NotFound>> (
        int id,
        IOrderService service,
        CancellationToken ct) =>
    {
        var order = await service.GetByIdAsync(id, ct);
        return order is null
            ? TypedResults.NotFound()
            : TypedResults.Ok(order);
    });
```

Route groups and extension methods

```
// Route groups reduce repetition:
var ordersGroup = app.MapGroup("/api/orders")
    .WithTags("Orders")
    .RequireAuthorization();

ordersGroup.MapGet("", GetAllOrders);
ordersGroup.MapGet("{id:int}", GetOrderById);
ordersGroup.MapPost("", CreateOrder);
ordersGroup.MapPut("{id:int}", UpdateOrder);
ordersGroup.MapDelete("{id:int}", DeleteOrder);

// Extension method -- keeps Program.cs clean:
public static class OrderEndpoints
{
    public static void MapOrderEndpoints(
        this WebApplication app)
    {
        var group = app.MapGroup("/api/orders")
            .WithTags("Orders");

        group.MapGet("", GetAll);
        group.MapGet("{id:int}", GetById);
        group.MapPost("", Create);
    }

    static async Task<Ok<PagedResult<OrderDto>>> GetAll(
        IOrderService svc, CancellationToken ct)
        => TypedResults.Ok(await svc.GetPagedAsync(ct));

    static async Task<Results<Ok<OrderDto>, NotFound>>
```

```

        GetById(
            int id,
            IOrderService svc,
            CancellationToken ct)
    {
        var o = await svc.GetByIdAsync(id, ct);
        return o is null
            ? TypedResults.NotFound()
            : TypedResults.Ok(o);
    }
}

// In Program.cs:
app.MapOrderEndpoints();

```

Controllers vs Minimal APIs

Use Controllers when:

- > Large APIs with many endpoints
- > Complex action filters needed
- > Team familiar with MVC patterns

Use Minimal APIs when:

- > Microservices or small focused APIs
- > Performance-critical (less overhead)
- > New greenfield projects

Key Takeaways for 13.3

- Minimal APIs define endpoints inline -- no controller classes needed
- Results.* / TypedResults.* -- response builders for Minimal APIs
- TypedResults preserves type information for Swagger documentation
- app.MapGroup() reduces route prefix repetition
- Extension method pattern keeps Program.cs clean for large apps

13.4. Model Binding and Validation

Builds on: 13.2 (controllers -- model binding feeds action parameters), 13.1 (routing -- route parameters are one binding source).

Model binding sources

```

// [FromRoute] -- from URL: GET /orders/42
[HttpGet("{id:int}")]
public IActionResult GetById([FromRoute] int id) { }

// [FromQuery] -- GET /orders?page=2&status=Pending
public IActionResult GetAll(
    [FromQuery] int page = 1,
    [FromQuery] string? status = null) { }

// [FromBody] -- from JSON body (POST/PUT):

```



```

[HttpPost]
public IActionResult Create(
    [FromBody] CreateOrderDto dto) { }

// [FromHeader] -- from HTTP header:
public IActionResult Create(
    [FromHeader(Name = "X-Correlation-Id")]
    string correlationId) { }

// [FromServices] -- from DI:
public IActionResult GetAll(
    [FromServices] IOrderService service) { }

// [FromForm] -- multipart/form-data (file uploads):
public IActionResult Upload(
    [FromForm] IFormFile file) { }

// With [ApiController], sources are inferred:
// Complex types (classes) -> [FromBody]
// Simple types (int, string) -> [FromRoute] then [FromQuery]
// IFormFile -> [FromForm]

```

Data annotations validation

```

public class CreateOrderDto
{
    [Required]
    [StringLength(100, MinimumLength = 2)]
    public string CustomerName { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    public string CustomerEmail { get; set; } = string.Empty;

    [Range(0.01, 1_000_000)]
    public decimal Amount { get; set; }

    [MinLength(1)]
    public List<OrderItemDto> Items { get; set; } = new();

    [Required]
    public AddressDto ShippingAddress { get; set; } = new();
}

// With [ApiController], invalid model returns automatically:
// 400 Bad Request with ValidationProblemDetails body:
// {
//     "title": "One or more validation errors occurred.",
//     "status": 400,
//     "errors": {
//         "CustomerName": ["The CustomerName field is required."],
//         "Amount": ["must be between 0.01 and 1000000."]
//     }
// }

```

FluentValidation -- complex validation rules

```
// dotnet add package FluentValidation.AspNetCore

public class CreateOrderDtoValidator
    : AbstractValidator<CreateOrderDto>
{
    public CreateOrderDtoValidator(
        ICustomerRepository customers)
    {
        RuleFor(x => x.CustomerName)
            .NotEmpty().HasMaxLength(100);

        RuleFor(x => x.Amount)
            .GreaterThan(0).LessThanOrEqualTo(1_000_000);

        RuleFor(x => x.Items)
            .NotEmpty()
            .WithMessage("Order must have at least one item");

        // Async rule -- checks database:
        RuleFor(x => x.CustomerId)
            .MustAsync(async (id, ct) =>
                await customers.ExistsAsync(id, ct))
            .WithMessage("Customer does not exist");
    }
}

// Register:
builder.Services.AddValidatorsFromAssemblyContaining<
    CreateOrderDtoValidator>();
```

Key Takeaways for 13.4

- [ApiController] infers binding sources -- complex types from body
- Data annotations validate at HTTP layer before action runs
- [ApiController] auto-returns 400 ValidationProblemDetails on failure
- FluentValidation for complex rules: async, conditional, database checks

Debugging Tips

Symptom	Cause	Fix
400 returned before action runs	[ApiController] auto-validation triggered by invalid model	Check DTO annotations; log ModelState.Values for specific errors
Body parameter always null	Content-Type header missing or wrong in request	Set Content-Type: application/json in client request
Query parameter not bound	Parameter name mismatch between query string and action param	Names are case-insensitive; check spelling matches exactly
Nested object not validated	Missing [Required] on nested DTO property	Add [Required] to nested object property in parent DTO

13.5. DTOs and Mapping

Builds on: 13.4 (model binding -- DTOs are what get bound), 9.1 (SRP -- DTOs separate API contract from domain model).

What are DTOs and why do they exist?

DTO (Data Transfer Object) = a class that carries data between layers. Without DTOs, exposing EF Core entities directly causes: circular reference errors, leaking internal data (passwords), API contract changing with DB schema, and over-fetching.

API (Controller) <-> DTO <-> Service <-> Entity <-> Database

DTO patterns

```
// Create DTO -- client sends to create:
public record CreateOrderDto(
    string CustomerName,
    string CustomerEmail,
    decimal Amount,
    List<OrderItemDto> Items);

// Update DTO -- client sends to update:
public record UpdateOrderDto(
    decimal Amount,
    string Status);

// Response DTO -- API returns:
public record OrderDto(
    int Id,
    string CustomerName,
    string Status,
    decimal Amount,
    DateTime CreatedAt,
    List<OrderItemDto> Items);

// Summary DTO -- for list endpoints (less data):
public record OrderSummaryDto(
    int Id,
    string CustomerName,
    string Status,
    decimal Amount,
    DateTime CreatedAt);
// No Items list -- reduces list payload size

// Paged response wrapper:
public record PagedResult<T>(
    IReadOnlyList<T> Items,
    int TotalCount,
    int Page,
    int PageSize)
{
    public int TotalPages =>
        (int)Math.Ceiling(TotalCount / (double)PageSize);
}
```

```

    public bool HasNextPage => Page < TotalPages;
    public bool HasPreviousPage => Page > 1;
}

```

Manual mapping -- clear and explicit

```

public static class OrderMappings
{
    public static OrderDto ToDto(this Order order) =>
        new(
            Id:            order.Id,
            CustomerName: order.CustomerName,
            Status:        order.Status,
            Amount:        order.Amount,
            CreatedAt:     order.CreatedAt,
            Items: order.Items
                .Select(i => i.ToDto()).ToList());

    public static OrderSummaryDto ToSummaryDto(
        this Order order) =>
        new(
            order.Id, order.CustomerName,
            order.Status, order.Amount, order.CreatedAt);

    public static Order ToEntity(
        this CreateOrderDto dto) =>
        new()
        {
            CustomerName = dto.CustomerName,
            CustomerEmail = dto.CustomerEmail,
            Amount        = dto.Amount,
            Status        = "Pending",
            CreatedAt     = DateTime.UtcNow,
            Items = dto.Items
                .Select(i => i.ToEntity()).ToList()
        };
}

```

EF Core projection -- most efficient mapping

```

// Project directly to DTO in SQL SELECT:
public async Task<List<OrderSummaryDto>> GetSummariesAsync(
    int userId, CancellationToken ct)
    => await _context.Orders
        .AsNoTracking()
        .Where(o => o.UserId == userId)
        .OrderByDescending(o => o.CreatedAt)
        .Select(o => new OrderSummaryDto(
            o.Id,
            o.CustomerName,
            o.Status,
            o.Amount,
            o.CreatedAt))
        .ToListAsync(ct);
// Only the 5 projected columns in SQL SELECT

```

```
// No Items list loaded -- much smaller payload
```

Key Takeaways for 13.5

- DTOs separate API contract from domain model -- essential for maintainability
- Separate Create/Update/Response DTOs -- don't share one DTO for everything
- Summary DTOs for list endpoints (fewer fields); full DTOs for single items
- Manual mapping with extension methods -- explicit, refactor-safe, no magic
- EF Core Select() projection -- most efficient, only fetches needed columns
- Never expose EF Core entities directly in API responses

13.6. Middleware Pipeline

Builds on: 12.6 (hosting -- middleware registered in Program.cs), 13.1 (routing -- routing is itself middleware).

What is middleware?

Middleware runs on every HTTP request and response. Each middleware can process the request, call next, and process the response on the way back. Together they form a pipeline:

```
Request:  ExceptionHandler->HTTPS->Auth->Routing->Controller
Response: Controller->Routing->Auth->HTTPS->ExceptionHandler
```

Custom middleware -- class-based approach

```
public class CorrelationIdMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<CorrelationIdMiddleware> _logger;

    public CorrelationIdMiddleware(
        RequestDelegate next,
        ILogger<CorrelationIdMiddleware> logger)
    { _next = next; _logger = logger; }

    public async Task InvokeAsync(HttpContext context)
    {
        var correlationId =
            context.Request.Headers["X-Correlation-Id"]
                .FirstOrDefault()
            ?? Guid.NewGuid().ToString();

        context.Items["CorrelationId"] = correlationId;
        context.Response.Headers
            ["X-Correlation-Id"] = correlationId;

        using (_logger.BeginScope(
            new { CorrelationId = correlationId }))
        {
            await _next(context);
        }
    }
}
```

```

    }
}

// Extension method:
public static class MiddlewareExtensions
{
    public static IApplicationBuilder UseCorrelationId(
        this IApplicationBuilder app)
        => app.UseMiddleware<CorrelationIdMiddleware>();
}

// Register:
app.UseCorrelationId();

```

Global exception handling middleware

```

public class GlobalExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<GlobalExceptionMiddleware> _log;

    public GlobalExceptionMiddleware(
        RequestDelegate next,
        ILogger<GlobalExceptionMiddleware> log)
    { _next = next; _log = log; }

    public async Task InvokeAsync(HttpContext context)
    {
        try { await _next(context); }
        catch (NotFoundException ex)
        {
            _log.LogWarning(ex, "Not found");
            await WriteProblem(context, 404, ex.Message);
        }
        catch (ValidationException ex)
        {
            _log.LogWarning(ex, "Validation failed");
            await WriteProblem(context, 400, ex.Message);
        }
        catch (Exception ex)
        {
            _log.LogError(ex, "Unhandled exception");
            await WriteProblem(context, 500,
                "An unexpected error occurred");
        }
    }

    private static async Task WriteProblem(
        HttpContext ctx, int code, string detail)
    {
        ctx.Response.StatusCode = code;
        ctx.Response.ContentType = "application/problem+json";
        await ctx.Response.WriteAsJsonAsync(
            new ProblemDetails

```

```

        { Status = code, Detail = detail });
    }
}

```

Key Takeaways for 13.6

- Middleware runs on every request in registration order -- order matters
- `app.Use(async (ctx, next) => ...)` -- inline middleware
- Class-based middleware with `InvokeAsync(HttpContext)` -- preferred for reuse
- Not calling `next` short-circuits the pipeline and returns immediately
- Exception handler middleware belongs first so it catches all errors

Debugging Tips

Symptom	Cause	Fix
Middleware not executing	Registered after <code>app.Run()</code> or short-circuit before it	Register all middleware before <code>app.Run()</code> ; check order
Authentication middleware has no effect	<code>UseAuthentication</code> after <code>UseAuthorization</code> -- wrong order	Always <code>UseAuthentication</code> before <code>UseAuthorization</code>
Response modified after headers sent	Writing to response after it has started streaming	Check <code>context.Response.HasStarted</code> before writing
Middleware throws but exception handler not catching	Exception handler not registered first in pipeline	Move <code>UseExceptionHandler</code> or custom exception middleware first

13.7. Filters

Builds on: 13.2 (controllers -- filters apply to actions), 13.6 (middleware -- filters are action-scoped middleware).

What are filters?

Filters run at specific points in the action execution pipeline -- after routing and model binding but before/after the action runs. Unlike middleware (runs for all requests), filters target specific controllers or actions.

```

Middleware pipeline
-> Routing
  -> Authorization filters
    -> Resource filters
      -> Model binding
        -> Action filters <- your logic here
          -> Action method
            -> Exception filters
              -> Result filters

```

Action filter -- before and after an action

```

public class ValidateOrderExistsAttribute
    : ActionFilterAttribute

```

```

{
    public override async Task OnActionExecutionAsync(
        ActionExecutingContext context,
        ActionExecutionDelegate next)
    {
        // Before action runs:
        var id = (int)context.ActionArguments["id"]!;
        var service = context.HttpContext.RequestServices
            .GetRequiredService<IOrderService>();

        if (!await service.ExistsAsync(id))
        {
            context.Result = new NotFoundObjectResult(
                $"Order {id} not found");
            return; // short-circuit -- action never runs
        }

        await next(); // execute the action
    }
}

// Apply to specific action:
[HttpDelete("{id:int}")]
[ValidateOrderExists]
public async Task<IActionResult> Delete(int id)
{
    await _service.DeleteAsync(id);
    return NoContent();
}

```

Exception filter -- handle specific exceptions

```

public class ApiExceptionHandlerAttribute
    : ExceptionFilterAttribute
{
    private readonly ILogger _logger;

    public ApiExceptionHandlerAttribute(
        ILogger<ApiExceptionHandlerAttribute> logger)
        => _logger = logger;

    public override void OnException(
        ExceptionContext context)
    {
        var (code, msg) = context.Exception switch
        {
            NotFoundException ex => (404, ex.Message),
            ForbiddenException ex => (403, ex.Message),
            ValidationException ex => (400, ex.Message),
            _ => (500, "Internal server error")
        };

        if (code == 500)
            _logger.LogError(context.Exception,
                "Unhandled exception");
    }
}

```



```

        context.Result = new ObjectResult(
            new ProblemDetails { Status = code, Detail = msg })
            { StatusCode = code };

        context.ExceptionHandled = true;
    }
}

// Register globally:
builder.Services.AddControllers(options =>
    options.Filters.Add<ApiExceptionHandlerAttribute>());

```

Key Takeaways for 13.7

- Filters run at specific points -- more targeted than middleware
- ActionFilterAttribute -- before/after action, can short-circuit
- ExceptionFilterAttribute -- handle specific exceptions from actions
- Global (options.Filters.Add), controller-level, or action-level scope

13.8. Content Negotiation, Status Codes, ProblemDetails

Builds on: 13.2 (controllers -- response methods), 13.4 (validation -- validation errors are ProblemDetails).

HTTP status codes -- the correct ones

GET	found	-> 200 Ok
GET	not found	-> 404 Not Found
POST	created	-> 201 Created + Location header
POST	invalid data	-> 400 Bad Request
PUT	updated	-> 204 No Content
PUT	not found	-> 404 Not Found
DELETE	deleted	-> 204 No Content
	Not logged in	-> 401 Unauthorized
	Logged in, no perm	-> 403 Forbidden
	Duplicate/conflict	-> 409 Conflict
	Business rule violated	-> 422 Unprocessable Entity
	Server error	-> 500 Internal Server Error

ProblemDetails -- RFC 7807 standard error format

```

// ASP.NET Core uses ProblemDetails for error responses:
// {
//   "type": "https://httpstatuses.com/404",
//   "title": "Not Found",
//   "status": 404,
//   "detail": "Order 42 was not found.",
//   "instance": "/api/orders/42",
//   "traceId": "00-abc123..."
// }

```

```
// Custom ProblemDetails:
return Problem(
    detail: "Order cannot be cancelled after shipping",
    statusCode: 422,
    title: "Invalid Operation",
    type:
        "https://api.myapp.com/errors/not-cancellable",
    instance: $"/api/orders/{id}");

// Enable ProblemDetails for all errors:
builder.Services.AddProblemDetails();

// Customise -- add traceId to every error response:
builder.Services.AddProblemDetails(options =>
    options.CustomizeProblemDetails = ctx =>
    {
        ctx.ProblemDetails.Extensions["traceId"] =
            Activity.Current?.Id ??
            ctx.HttpContext.TraceIdentifier;
        ctx.ProblemDetails.Extensions["nodeId"] =
            Environment.MachineName;
    });
```

Key Takeaways for 13.8

- Use correct HTTP status codes -- 201 for Create, 204 for Update/Delete
- ProblemDetails (RFC 7807) -- standard error format in .NET APIs
- `builder.Services.AddProblemDetails()` -- enables for all error responses
- Customise ProblemDetails to add traceId and diagnostic fields

13.9. API Versioning | 13.10. Swagger | 13.11. Health Checks | 13.12. Rate Limiting

Builds on: 12.2 (DI -- all registered on `builder.Services`), 12.6 (middleware -- all added to the pipeline).

13.9. API Versioning

```
// dotnet add package Asp.Versioning.Mvc

builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion =
        new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
    options.ApiVersionReader =
        ApiVersionReader.Combine(
            new QueryStringApiVersionReader("api-version"),
            new HeaderApiVersionReader("X-API-Version"),
            new UrlSegmentApiVersionReader());
```

```

});

// Version via URL segment:
[ApiController]
[Route("api/v{version:apiVersion}/orders")]
[ApiVersion("1.0")]
[ApiVersion("2.0")]
public class OrdersController : ControllerBase
{
    [HttpGet]
    [MapToApiVersion("1.0")]
    public IActionResult GetV1() => Ok("v1 response");

    [HttpGet]
    [MapToApiVersion("2.0")]
    public IActionResult GetV2() => Ok("v2 response");
}

```

13.10. Swagger / OpenAPI

```

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title      = "MyApp API",
        Version    = "v1",
        Description = "Order management API"
    });

    // Add JWT auth to Swagger UI:
    options.AddSecurityDefinition("Bearer",
        new OpenApiSecurityScheme
        {
            Type = SecuritySchemeType.Http,
            Scheme = "bearer"
        });
});

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(c => c.SwaggerEndpoint(
        "/swagger/v1/swagger.json", "MyApp API v1"));
}

// XML doc comments on action methods:
/// <summary>Get an order by ID</summary>
/// <response code="200">Order found</response>
/// <response code="404">Order not found</response>
[HttpGet("{id:int}")]
[ProducesResponseType<OrderDto>(200)]
[ProducesResponseType(404)]
public async Task<ActionResult<OrderDto>> GetById(int id) {}

```

13.11. Health Checks

```
builder.Services.AddHealthChecks()
    .AddSqlServer(
        connectionString: connStr,
        name: "database",
        tags: ["db", "sql"])
    .AddUrlGroup(
        new Uri("https://api.payment.com/health"),
        name: "payment-api",
        tags: ["external"])
    .AddCheck<OrderQueueHealthCheck>(
        "order-queue", tags: ["queue"]);

app.MapHealthChecks("/health");

// Tag-based filtering:
app.MapHealthChecks("/health/ready",
    new HealthCheckOptions
    {
        Predicate = c => c.Tags.Contains("db")
    });

// Custom health check:
public class OrderQueueHealthCheck : IHealthCheck
{
    public Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext ctx,
        CancellationToken ct = default)
    {
        var depth = GetQueueDepth();
        return Task.FromResult(
            depth < 1000
                ? HealthCheckResult.Healthy(
                    $"Queue: {depth}")
                : HealthCheckResult.Degraded(
                    $"Queue full: {depth}"));
    }
}
```

13.12. Rate Limiting

```
// .NET 7+ built-in rate limiting:
builder.Services.AddRateLimiter(options =>
{
    // Fixed window -- 100 req/min per IP:
    options.AddFixedWindowLimiter("fixed", lim =>
    {
        lim.Window      = TimeSpan.FromMinutes(1);
        lim.PermitLimit = 100;
        lim.QueueLimit  = 10;
    });

    // Token bucket -- burst allowance:
    options.AddTokenBucketLimiter("token", lim =>
```

```

{
    lim.TokenLimit          = 20;
    lim.TokensPerPeriod     = 5;
    lim.ReplenishmentPeriod =
        TimeSpan.FromSeconds(1);
});

options.RejectionStatusCode = 429;
options.OnRejected = async (context, ct) =>
{
    context.HttpContext.Response.StatusCode = 429;
    await context.HttpContext.Response
        .WriteAsJsonAsync(
            new ProblemDetails
            {
                Status = 429,
                Title  = "Too Many Requests"
            }, ct);
};
});

app.UseRateLimiter();

// Apply to specific action:
[HttpPost]
[EnableRateLimiting("fixed")]
public async Task<IActionResult> Create(
    CreateOrderDto dto) { }

// Apply globally:
app.MapControllers().RequireRateLimiting("fixed");

```

Key Takeaways for 13.9 - 13.12

- 13.9 API Versioning: URL segment, query string, or header versioning strategies
- 13.10 Swagger: AddSwaggerGen + UseSwagger in Development; XML comments for docs
- 13.11 Health checks: AddHealthChecks + AddSqlServer/AddUrlGroup/custom IHealthCheck
- 13.12 Rate limiting: AddRateLimiter with Fixed/Sliding/TokenBucket policies (.NET 7+)

Debugging Tips (13.9-13.12)

Symptom	Cause	Fix
Swagger shows no endpoints	AddEndpointsApiExplorer() not called or Swagger not in Development	Add AddEndpointsApiExplorer(); wrap UseSwagger in IsDevelopment()
Health check always Healthy despite DB down	AddSqlServer not installed or wrong connection string	dotnet add package.AspNetCore.HealthChecks.SqlServer; verify connstr
Rate limiter returns 503 not 429	RejectionStatusCode not set	Set options.RejectionStatusCode = 429 in AddRateLimiter

Symptom	Cause	Fix
API version not resolved	Version reader not configured for the client's method	Add all three readers: QueryString, Header, UriSegment

Section 13 -- Complete Key Takeaways

Topic	Core concept	Key API
-----+-----+-----		
13.1 Routing	Attribute + constraints	[HttpGet]{id:int}
13.2 Controllers	ControllerBase+results	Ok(),NotFound()
13.3 Minimal API	Endpoints no class	app.MapGet()
13.4 Binding	Auto from route/body	[Required],[Range]
13.5 DTOs	API contract separate	Manual mapping
13.6 Middleware	Cross-cutting pipeline	app.Use()
13.7 Filters	Action-scoped concerns	ActionFilterAttr
13.8 ProblemDet.	RFC 7807 error format	AddProblemDetails
13.9 Versioning	Multi-version support	AddApiVersioning
13.10 Swagger	API documentation	AddSwaggerGen
13.11 Health	Dependency checks	AddHealthChecks
13.12 RateLimit	Protect from abuse	AddRateLimiter

Section 13 complete. All 12 roadmap topics (13.1 through 13.12) covered. Say next to begin Section 14: Entity Framework Core.